Two-Microphone Acoustic Imaging on FPGA: Code Explained for the Project Review

Oct 29, 2026 · @jovin sebastian

The project listens with two digital MEMS microphones, works out which side a sound comes from by comparing tiny arrival-time differences, and paints that as a colour image on a VGA monitor, all inside the Cyclone IV FPGA on a DE2-115 board.

The big picture

The design is a chain of blocks that all run at the same time inside the FPGA: it reads the mics, turns their bits into audio, finds the sound's direction, and draws it.

Sound flows down this chain from the mics to the screen



system block diagram · 8 stages, 2 side outputs

Read it top to bottom. The highlighted delay-and-sum block is the heart of the project: it is where the direction is found. The right-hand branch measures loudness and frequency for the panel and the LEDs.

Key ideas in plain words

Six ideas explain the whole design; learn these first and the code reads like a list of steps.

1. PDM microphones give 1 bit at a time. The Adafruit mic (MP34DT01-M) has no analog output. The FPGA sends it a clock (3.125 MHz) and on every tick it answers with one bit, 1 or 0. Loud positive pressure gives mostly 1s, negative pressure mostly 0s, silence about half and half. This is *pulse density modulation*: the sound is hidden in the density of 1s.

2. Two mics share one wire. Each mic has a SEL pin. Mic 1 (SEL = GND) puts its bit on the DAT wire while the clock is high; Mic 2 (SEL = 3.3 V) puts its bit on the same wire while the clock is low. The FPGA reads the wire once in each half and so gets both mics from one pin.

3. Decimation turns millions of bits into normal audio. Averaging the bits gives a normal audio number (PCM). A CIC filter does this cheaply: it keeps running sums and outputs one 16-bit sample for every 16 bits, so 3.125 MHz becomes 195.3 kHz (195,312 samples per second). Extra small filters remove DC offset, high-frequency noise and low hum.

4. Delay-and-sum finds the direction. Sound travels 343 m/s, so a sound from the left reaches Mic 1 slightly before Mic 2 (up to 89 µs for mics 30.5 mm apart, about 17 samples). The design tries 33 possible delays. For each one it delays Mic 1, adds the two signals and measures the energy. When the trial delay matches the real arrival difference the two waves line up and add up strongly; otherwise they partly cancel. The delay with the biggest energy tells the direction. This is called *beamforming*.

5. Decibels and a log scale make the picture readable. Energy can vary a million times between a whisper and a shout. The design takes log2 of the energy (each step of 1 = 3 dB) and colours each direction relative to the loudest one: within 3 dB of the loudest is red, further down goes yellow, green, then blue.

6. VGA draws the screen pixel by pixel. A VGA monitor expects 640 x 480 pixels sent left to right, top to bottom, 60 times per second (a 25 MHz pixel rate) with two timing pulses (HSYNC at the end of each line, VSYNC at the end of each frame). The VGA module keeps two counters for the current pixel position and, for each position, decides the colour: border, text, panel or image.

The files and what each one does

The top module is the "main program"; it creates (instantiates) the other modules and wires them together.

File	Role in one line
de2_115_two_mic_grid_top.v	Top level: makes the mic clock, reads both mics, runs the direction sweep, builds the image, drives LEDs and 7-segment displays
pdm_channel.v	One microphone channel: 1-bit PDM to 16-bit audio (CIC + filters), plus health check and loudness level. Used twice
mic_ring_buffer_1k.v	A memory that stores the last 1024 audio samples of one mic, so past samples can be read back with a delay. Used twice
freq_counter_hz.v	Measures the frequency of the sound by counting wave cycles in exactly one second. Used twice
vga_2mic_grid.v	Draws the 640 x 480 screen: the 1x2 image box, corner tags, SOURCE DATA panel and the colour legend
tb_two_mic_grid.v	Testbench for ModelSim: fake microphones play a 3 kHz tone from the left, then the right, then silence
pin_assignments_2mic_grid.tcl	Tells Quartus which physical FPGA pin each signal uses
two_mic_grid.sdc	Tells the timing analyser the clock is 50 MHz and which paths need no timing check

Verilog words used below: a **module** is a hardware block; **reg** is a storage element (flip-flop) that changes only on a clock edge; **wire** is a connection; `always @(posedge CLOCK_50)` means "do this on every rising edge of the 50 MHz clock"; `<=` is a register update that takes effect at that edge; `assign` is permanent combinational logic; `generate for` copies a block of hardware several times.

Top module walkthrough (de2_115_two_mic_grid_top.v)

The top file is split into nine numbered sections that run in parallel all the time; data flows from section 1 to section 8.

Parameters and ports. `GATE_SAMPLES = 195312` is one second of samples for the frequency counter. `GATE_BASE = 15` is the default noise gate. `WF_SHIFT = 6` means one new image row every 2^6 = 64 sweeps. The ports are the board's real pins: `CLOCK_50`, the reset button `KEY[0]`, the 18 switches `SW`, the mic clock and data pins, the VGA pins, LEDs and the eight 7-segment displays. `rst_n = KEY[0]` is an active-low reset: pressing KEY0 clears every register.

Section 1: mic clock and capture. A 4-bit counter `div` counts 0 to 15 at 50 MHz. Its top bit `div[3]` is low for 8 counts and high for 8, so it is a 50/16 = 3.125 MHz square wave, sent to both mics on `GPIO_MIC_CLK`. The DAT pin first passes through two flip-flops (`d0_s1`, `d0_s2`). This *2-flop synchronizer* stops an input that changes at a bad moment from upsetting the logic (metastability). When `div == 7` (end of the clock-low half) the bit belongs to Mic 2 and is saved in `r0`. When `div == 15` (end of the clock-high half) the bit belongs to Mic 1; both bits are then stored as `pdm_bits = {Mic 2, Mic 1}` and a one-cycle pulse `pdm_ce` ("clock enable") says "new bits ready". Reading near the end of each half gives the mic time to drive the wire, which is why this method worked reliably on the board.

Section 2: two channels. A `generate for` loop creates two copies of `pdm_channel`, one per mic. Each gives `audio` (clean audio for level and frequency), `audio_hp` (audio with low frequencies removed, for direction finding), `audio_ce` (a pulse at 195.3 kHz when a new sample is ready), `density` (health check) and `level` (loudness). `sample_ce` is simply channel 0's sample pulse; both channels pulse together.

Section 3: ring buffers and the direction sweep. `wr_ptr` counts up on every new sample and is the write address of two 1024-sample memories, one per mic. The sweep counters are `scan_cell` (direction 0 to 32, left to right) and `lidx` (sample 0 to 511). For each direction the design reads 512 past samples, then moves to the next direction; after direction 32 it starts again and freezes `base = wr_ptr` as the starting point for the new sweep. For each direction, `tau1 = 16 - dir` is the trial delay for Mic 1, from +16 samples (far left) to -16 (far right). The read address for Mic 2 is `rd_base = base - 20 - lidx`; Mic 1 is read `tau1` samples earlier (`rd1`). The 20-sample offset makes sure every read is in the past. `SW[16]` mirrors left and right.

Section 4: delay-and-sum energy. The two delayed samples are added (`bsum = dly0 + dly1`) and squared (`bsq`), because energy is amplitude squared. The square is added into `acc` for all 512 samples of a direction. On the last sample (`p2_last`) the total becomes `cell_energy`, `e_valid` pulses, and `acc` is cleared for the next direction. The `p1_` and

`p2_` registers are a *pipeline*: they delay the "last sample" flag and the direction number by the same number of clocks as the data, so they stay lined up. One sweep of 33 directions takes 33 x 512 = 16,896 clocks = 0.34 ms.

Section 5: log, smoothing, intensity, peak and nearest mic. This is the "brain" and runs once per direction:

1. **log2.** `msb44` finds the highest 1 bit of the 44-bit energy (that is the integer part of log2). Three more bits below it (`mant_sh`) give a fraction, so `lg2` is log2(energy) with 1/8 steps. One step of 1.0 = 3 dB, one step of 1/8 = 0.38 dB.
2. **Smoothing.** `smooth_ram` keeps one value per direction. Each sweep it moves 1/64 of the way to the new value: `sm += (new - sm) / 64`. This removes flicker; the time constant is about 64 sweeps = 22 ms.
3. **Colour window.** `SW[1:0]` sets how many dB below the loudest direction still get colour: 3 dB (default), 6, 12 or 1.5 dB. `rel` is how far this direction is inside that window; `inten` (0 to 255) is the colour value: 255 = as loud as the loudest (red), 0 = more than the window below it (blue).
4. **Noise gate.** `gate_oct = 15 + SW[5:2]`. If the loudest direction is below the gate, `prev_active = 0` and the image is all blue, so room noise is not shown as a source.
5. **Peak.** `cur_max` tracks the loudest direction. Because two close mics give a very flat beam, many directions tie; the code therefore takes the first and last direction within 0.75 dB of the maximum (`cur_first`, `cur_last`) and uses the middle (`pk`).
6. **Nearest mic.** At the end of each sweep: peak left of column 15 means the sound reached Mic 1 first, so `nearest = 1`; right of column 17, `nearest = 2`; in between, 3 (EQ, roughly straight ahead); below the gate, 0.
7. **dir_int.** The colour value of each of the 33 directions is stored in `dir_int`.

Section 6: history image writer. Every 64 sweeps (about 22 ms) `wf_busy` copies the 33 values of `dir_int` into one row of the image memory inside the VGA module and advances `wf_row`. `newest_row` tells the VGA module which row is newest. `SW[17]` up stops the copying, which freezes the picture.

Section 7: frequency. Two `freq_counter_hz` blocks measure the frequency of each mic once per second, as four BCD digits in Hz.

Section 8: VGA. One `vga_2mic_grid` instance receives the image rows, the levels, frequencies, nearest mic, active flag and peak column, and drives the VGA pins.

Section 9: LEDs and 7-segment displays. LEDR7–0 and LEDR15–8 are level bars for Mic 1 and Mic 2. LEDG0 and LEDG1 light when each mic's 1s-density is healthy (0x60 to 0xA0, near 50 %). LEDG8 shows the image is active. The function `seg7` converts a 4-bit number into the seven segment pattern (segments are active-low: 0 = on). HEX7–6 and HEX5–4 show the two levels, HEX3–0 the frequency of the nearest mic.

pdm_channel.v: turning 1-bit mic data into audio

This module takes one mic's bit stream (3.125 million bits per second) and turns it into a clean 16-bit audio signal at 195,312 samples per second, in five small steps.

Step 1: health check (density). For every 1024 bits it counts the 1s (`ones`) and reports `density = ones / 4` . A working mic in a quiet room gives about 50 % 1s, so density about 0x80 (128). Stuck at 0x00 means the data wire is always low (no clock, no power, wrong pin); 0xFF means stuck high. The top module lights LEDG0/LEDG1 when density is between 0x60 and 0xA0.

Step 2: CIC decimator (bits to samples). Each bit is turned into +1 or -1 (`d_in`). Three *integrators* (`int1` , `int2` , `int3`) are running sums: `int1` adds the input, `int2` adds `int1` , `int3` adds `int2` . Every 16th bit (`dec_cnt == 15`) three *comb* stages take differences (`comb = now - previous`). Integrate three times, then difference three times every 16 inputs: that equals averaging over a 16-bit window three times in a row, which is a good low-pass filter that needs only adders, no multipliers. The output rate is 3.125 MHz / 16 = 195.3 kHz. `cic_ce` pulses once per output sample. The output is shifted left by 2 (`raw_pcm`) so full scale is about ±16,384.

Step 3: DC blocker. `dc_est` is a slow running average of the signal (it moves 1/256 of the way each sample). Subtracting it (`pcm = raw_pcm - dc_est`) removes any constant offset, leaving only the sound.

Step 4: two-pole low-pass filter. `lp1 += (input - lp1) / 8` and `lp2 += (lp1 - lp2) / 8` . Each line is a simple first-order low-pass with a corner near 3.9 kHz; two in a row remove the high-frequency hiss that PDM produces above the audio band. The result is `audio` , used for the level meter and frequency counter. The `>>> 3` means divide by 8 keeping the sign.

Step 5: high-pass for direction finding. `hp_lp` is another slow average (1/32 step, corner near 1 kHz); `audio_hp = audio - hp_lp` keeps only frequencies above about 1 kHz. Low frequencies (fans, hum, most room noise) give no direction information with mics only 3 cm apart, so they are removed before the beamformer.

Level meter. `mag` is the absolute value of `audio`. `peak` jumps up to any bigger value and slowly decays (1/8 every 4096 samples, about 21 ms). `msb_pos16` returns the position of the highest 1 bit of the peak, which is log2 of the amplitude: one step = 2x amplitude = 6 dB. Silence gives about 2, speech 5–7, a loud tone 9–11. This is the LVL value on the screen.

Why clock enables: everything runs on the single 50 MHz clock and the slower rates use one-cycle enable pulses (`pdm_ce`, `cic_ce`, `audio_ce`). That avoids extra clock domains, which are a common source of FPGA timing bugs.

mic_ring_buffer_1k.v and freq_counter_hz.v

mic_ring_buffer_1k.v: the audio memory

A ring buffer is a memory used like a clock face: the write address goes 0, 1, 2 … 1023 and wraps back to 0, always overwriting the oldest sample. It holds the last 1024 samples (5.2 ms) of one mic.

- `mem` is 1024 words of 16 bits. Quartus builds it from the FPGA's block RAM (M9K blocks), not from logic.
- On every clock: if `we` (write enable) is high, the new sample is written at `wr_addr`; and the sample at `rd_addr` is read into `dout` (available one clock later).
- Reading at "write pointer minus N" gives the sample from N samples ago. That is how the top module creates the trial delays for beamforming: Mic 1 is simply read from a different (earlier or later) address than Mic 2.

freq_counter_hz.v: measuring frequency

It counts how many times the sound wave goes up through zero in exactly one second; that count is the frequency in Hz.

- **Peak tracker.** Same idea as the level meter: `peak` follows the loudest recent value.
- **Hysteresis.** A crossing is only counted when the signal rises above +H after being below -H, with H = peak / 4 (at least 8). This ignores small noise wiggles around zero, which would otherwise be counted as extra cycles. Scaling H with the peak makes it work for quiet and loud sounds.
- **Loudness check.** Nothing is counted if the peak is below 32, so silence reads 0000 instead of random numbers.
- **Gate.** `gate_cnt` counts 195,312 samples = exactly 1.000 s. At the end the count is copied to `freq_bcd` and restarted. So the display updates once per second.

- **BCD counting.** `bcd_inc` adds 1 in *binary-coded decimal*: each 4-bit group holds one decimal digit 0–9 (when a digit passes 9 it becomes 0 and carries). This makes the digits ready for the display without a binary-to-decimal converter. The screen shows `freq_bcd[15:12]` (thousands = kHz) and `freq_bcd[11:8]` (hundreds = tenths of kHz), e.g. 3.0 KHZ.

vga_2mic_grid.v: how the screen is drawn

The module never stores a whole picture; for every pixel position it calculates the colour on the fly, 25 million times a second.

Pixel clock and counters. `clk_vga` toggles every 50 MHz clock, giving 25 MHz; `pix_ce` is the enable for "next pixel". Counter `h` runs 0–799 across a line and `v` runs 0–524 down the screen; only h < 640 and v < 480 is visible, the rest is blanking time the monitor needs. 800 x 525 x 60 ≈ 25 million pixels per second.

Sync pulses. `vga_hs` goes low for h = 656–751 (end of each line) and `vga_vs` goes low for v = 490–491 (end of each frame). This is the standard 640 x 480 @ 60 Hz timing. `vga_blank_n` is high only in the visible area.

Text. The screen is treated as a grid of 8 x 8 pixel cells: `tcol = h / 8` and `trow = v / 8`. The big `case` block (marked GENERATED) says which character sits in each cell, for example row 7, columns 59–69 spell "SOURCE DATA", and the NEAR, FREQ, QUAD and LVL fields pick their characters from the live values. The function `glyph` is a small font: for each character code and pixel row it returns 5 bits, one per pixel column (a 5 x 7 font). `text_on` is 1 when the current pixel is a lit part of a letter. This was produced by a Python script so text and font stay consistent; it is ordinary Verilog.

Number formatting. `fsel` is the frequency of the nearest mic. `l1_t` / `l1_o` split a level 0–16 into tens and ones digits. `quad` decides the QUAD text: CH 01, CH 02, BOTH or --.

The image box. The box is 384 x 384 pixels at x 64–447, y 48–431, split by a white line at x = 256 into the Mic 1 half and the Mic 2 half. Horizontally, `xs = rel_x x 88` maps 384 pixels onto the 33 directions (`icol`), and `frac` is the position between two directions. Vertically, every 8 pixels is one history row (`rvis` , 48 rows); row 0 at the top is the newest. `wrow = newest_row - rvis` finds that row in the memory.

History memory. `wf_ram_a` and `wf_ram_b` are identical 4096-byte memories (64 rows x 64 columns). The top module writes a new row every 22 ms. Two copies exist so two neighbouring directions can be read at the same time for smoothing.

Smoothing between directions. `lerp(a, b, f)` blends the two neighbouring values: `(a x (16 - f) + b x f) / 16`. This turns 33 blocky columns into a smooth gradient.

Colour map. `heat(s)` turns the value 0–255 into red, green and blue: 0–63 blue to cyan, 64–127 cyan to green, 128–191 green to yellow, 192–255 yellow to red. So high intensity = red, middle = yellow/green, low = blue.

Corner tags. Two 32 x 36 boxes at the top-left of each half show "1" and "2". `hl1` / `hl2` turn the nearest mic's tag amber (both when EQ).

Panel and legend. `in_hud` is the SOURCE DATA panel with a cyan border; `in_leg` draws the vertical INTENSITY legend, which is just the `heat` colours from 255 at the top to 0 at the bottom with HIGH / MID / LOW labels. The legend is a fixed key, not a measurement.

Output multiplexer. The `pix` block is a priority list: borders and grid line first, then tags, then the image, the panel border, text, panel background, legend, and finally the dark background. On every pixel enable the chosen colour is copied to `vga_r`, `vga_g`, `vga_b` (8 bits each) for the board's ADV7123 video DAC, which converts them to the analog VGA voltages.

Testbench, TCL pin file and SDC timing file

tb_two_mic_grid.v (simulation in ModelSim)

A testbench is Verilog that is never put on the FPGA; it creates fake inputs and checks the outputs in simulation.

- `pdm_mic_model` is a software model of the real microphone. It generates a sine wave, converts it into 1-bit PDM with a *second-order sigma-delta modulator* (the same principle the real mic uses), and drives the shared DAT wire only in its own clock half, like a real SEL = GND or SEL = 3.3 V mic. The `tri0` wire models the 47 kohm pull-down.
- Each model computes its own arrival delay from its position (±15.25 mm) and the source direction, so the testbench reproduces the real time difference between the two mics.
- The test: reset, then a 3 kHz tone from 50° on Mic 1's side for 40 ms, then from Mic 2's side for 40 ms, then silence for 25 ms.
- Expected printout: peak column 0–10 and nearest 1, then column 22–32 and nearest 2, then active 0. To make simulation faster, the frequency gate is 50 ms instead of 1 s, so frequencies read 1/20 (3000 Hz shows as 0150), and history rows are written every 4 sweeps.

pin_assignments_2mic_grid.tcl

A script for Quartus (Tools > Tcl Scripts) that connects each signal name in the top module to a physical FPGA pin, for example `CLOCK_50` to PIN_Y2, `GPIO_MIC_CLK` to PIN_AB22 (JP5 pin 1), `GPIO_MIC_DAT0` to PIN_AB21 (JP5 pin 3), plus the VGA, switch, LED and 7-segment pins. It also sets each pin's voltage standard (3.3 V or 2.5 V) to match the board.

two_mic_grid.sdc

Timing constraints for the Timing Analyzer. `create_clock` says CLOCK_50 has a 20 ns period, so Quartus checks that every register-to-register path finishes in under 20 ns. `set_false_path` lines tell it not to time the slow or asynchronous pins (buttons, switches, mic data that is already synchronised, LEDs, VGA outputs). Because the whole design runs on one clock, no clock-crossing constraints are needed.

Key numbers to remember

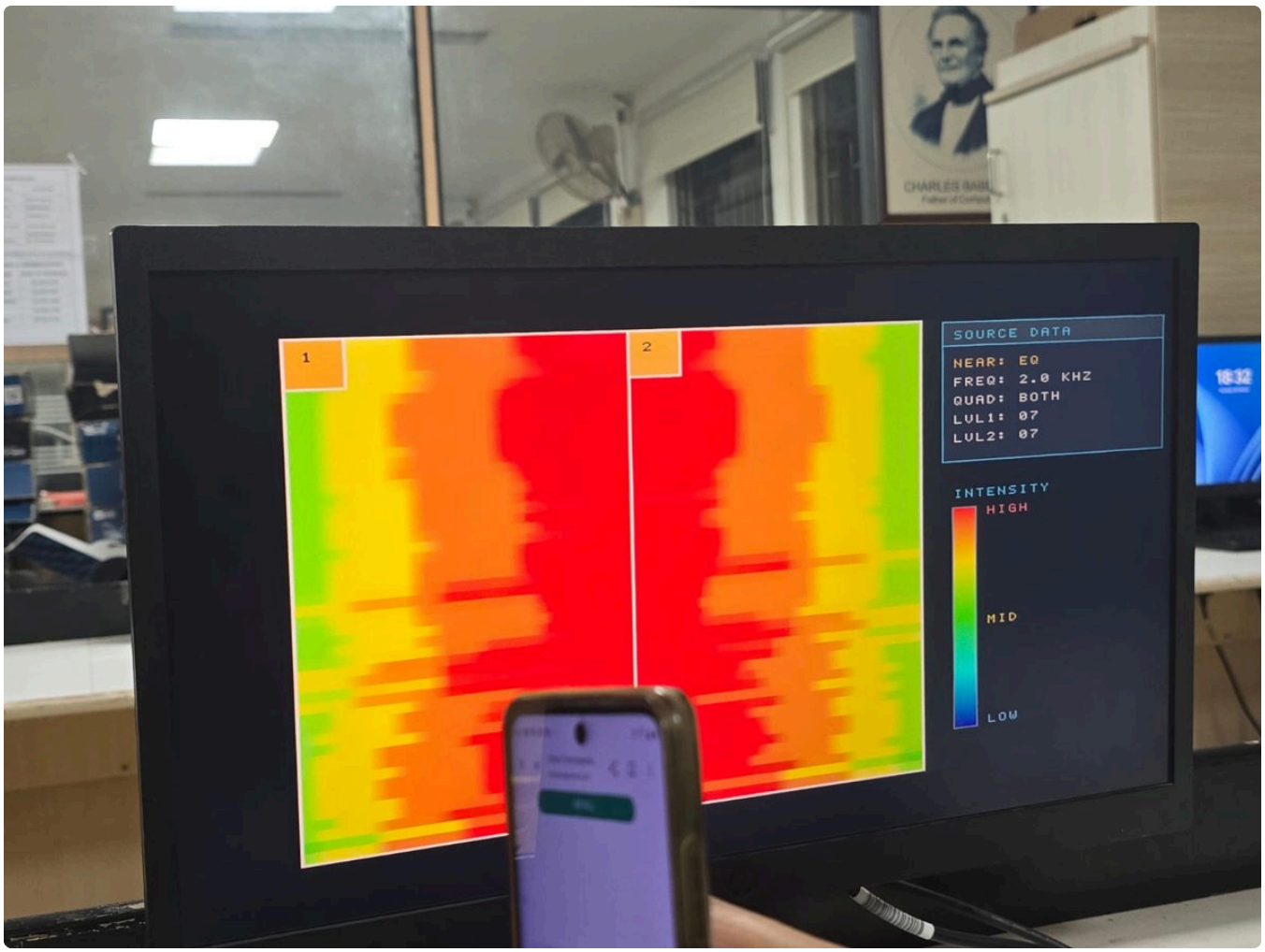
Examiners often ask for numbers; these all come straight from the code.

Quantity	Value	Where it comes from
System clock	50 MHz	DE2-115 oscillator, CLOCK_50
Mic clock	3.125 MHz	50 MHz / 16 (`div[3]`)
Audio sample rate	195,312.5 samples/s	3.125 MHz / 16 (CIC decimation R = 16)
CIC filter	3rd order, R = 16	three integrators + three combs
Low-pass corner	about 3.9 kHz per pole, 2 poles	`>>> 3` filters
High-pass (direction path)	about 1 kHz	`>>> 5` filter
Mic spacing	30.5 mm (12 breadboard rows)	wiring guide
Largest arrival difference	89 µs = 17.4 samples	30.5 mm / 343 m/s

Quantity	Value	Where it comes from
Trial delays	33 directions, -16 to +16 samples	`tau1 = 16 - dir`, about ±67°
Samples per direction	512 (2.6 ms of audio)	`lidx` 0..511
One full sweep	0.34 ms	33 x 512 clocks at 50 MHz
log scale	1 step = 3 dB, 1/8 step = 0.38 dB	`lg2`
Smoothing	about 22 ms	1/64 per sweep
Colour window (default)	3 dB below the loudest direction	`SW[1:0] = 00`
Noise gate (default)	15	`GATE_BASE` + `SW[5:2]`
Image	33 directions x 48 rows, new row every 22 ms (about 1 s shown)	`WF_SHIFT = 6`
Screen	640 x 480 at 60 Hz, 25 MHz pixel rate	800 x 525 total
Frequency measurement	1 s gate, shown in kHz with 1 decimal	`freq_counter_hz`
Level	1 step = 6 dB	log2 of peak amplitude

How to present it in the review

Tell it as a story of the sound's journey, from the air to the screen, in about 8 minutes, then show it working.



Talk order (one slide or whiteboard step each):

1. **Aim (30 s).** "We built a real-time acoustic imaging system on the DE2-115 FPGA. Two digital MEMS microphones listen, the FPGA works out which side the sound comes from, and a VGA monitor shows it as a colour image, with the frequency, level and nearest microphone."
2. **Hardware (1 min).** Two Adafruit PDM mics 30.5 mm apart on a breadboard, sharing one clock and one data wire; Mic 1 SEL to GND, Mic 2 SEL to 3.3 V, so they take turns on the wire. 33 ohm series resistor on the clock, 47 kohm pull-down on data, decoupling capacitors. Show the wiring schematic.
3. **Reading the mics (1 min).** The FPGA generates a 3.125 MHz clock; each mic returns 1 bit per tick; the FPGA samples the shared wire late in each clock half, so it separates the two mics. A 2-flop synchronizer protects against metastability.
4. **PDM to audio (1 min).** A third-order CIC decimator averages the bits down to 195 kHz, 16-bit audio. A DC blocker and two-pole low-pass clean it. Each mic also gets a level meter and a frequency counter.

5. **Finding the direction (2 min, the key part).** Explain delay-and-sum with the block diagram: the sound reaches the nearer mic first, by up to 89 µs. The FPGA stores the last 1024 samples of each mic in block RAM, tries 33 delays from -16 to +16 samples, adds the two signals and measures the energy over 512 samples. The direction where the energy is largest is where the sound came from. All 33 directions are tested every 0.34 ms.
6. **From energy to colour (1 min).** log2 of energy (3 dB steps), smoothing over 22 ms, colour relative to the loudest direction (red within 3 dB, then yellow, green, blue), and a noise gate so silence stays blue. Nearest mic: peak on the left half = Mic 1, right half = Mic 2, centre = EQ.
7. **Display (1 min).** VGA 640 x 480 at 60 Hz generated in logic: pixel counters, sync pulses, a text engine with a 5 x 7 font for the SOURCE DATA panel, a 1x2 image box (left half Mic 1 side, right half Mic 2 side), rows scrolling as time, a colour legend.
8. **Verification (30 s).** ModelSim testbench with modelled PDM mics and a moving 3 kHz source: the peak moves left then right and the image turns off in silence. Then single-mic bring-up on the board using the LEDs and 7-segment displays before the full system.
9. **Result and limits (30 s).** Show the photo or the live demo, then mention the limits honestly (see the last section).

Live demo steps:

1. Program the board, connect the VGA monitor, keep SW17 down and SW0–SW5 down.
2. In silence: the box is blue, NEAR shows --, LVL1/LVL2 about 02. Point out LEDG0 and LEDG1 (both mics healthy).
3. Play a steady 3–4 kHz tone from a phone app 20–30 cm away, far to Mic 1's side: FREQ shows the tone, the left half turns red, tag 1 turns amber, NEAR M01.
4. Move the phone slowly to the other side: the red band moves across and leaves a trail in the scrolling rows; NEAR changes to M02.
5. Hold it straight ahead: red in the centre, both tags amber, NEAR EQ (as in the photo).

Tips: use 3–4 kHz, not low tones (low tones look the same from every direction with mics this close); keep the phone level with the mics; if the room is noisy and the image flickers in silence, raise SW2–SW5 to raise the noise gate.

Viva questions and answers

Short, honest answers in your own words score better than memorised ones; these cover the questions examiners usually ask.

Q1. What is a PDM microphone? A digital MEMS mic with a built-in sigma-delta modulator. It outputs 1 bit per clock tick; the density of 1s represents the sound pressure. It needs an external clock (1–3.25 MHz); we use 3.125 MHz.

Q2. How do two mics share one data wire? With the SEL pin. SEL = GND makes the mic drive the wire while the clock is high; SEL = 3.3 V makes it drive while the clock is low. The other half it goes high-impedance. The FPGA reads once in each half, so it gets both mics.

Q3. Why sample at div = 7 and div = 15, not on the clock edges? The mic needs time after each clock edge to put its bit on the wire. Reading at the end of each half gives the data the most time to settle. Our first 4-mic version sampled on the clock edges (and also had a pin mismatch) and showed nothing; the single-mic test that samples late in each half worked first time, so we kept that method.

Q4. What is the 2-flop synchronizer for? The DAT signal comes from outside and can change at any moment relative to the FPGA clock. A flip-flop sampling a changing input can become metastable; passing it through two flip-flops gives it time to settle before the logic uses it.

Q5. What does the CIC filter do and why use it? It is a low-pass filter plus decimator built only from adders and registers: three integrators run at the high rate, three combs at the low rate. It averages the 1-bit stream and outputs one 16-bit sample every 16 bits. It needs no multipliers, so it is cheap on an FPGA.

Q6. Why are there extra filters after the CIC? The CIC alone leaves a lot of high-frequency PDM noise and a DC offset. The DC blocker removes the offset, the two-pole low-pass (about 3.9 kHz) removes hiss, and a high-pass (about 1 kHz) removes low-frequency room noise before direction finding.

Q7. Explain delay-and-sum beamforming. Sound from one side reaches the nearer mic first. If we delay that mic's signal by the right amount, the two signals line up and add in phase, giving maximum energy. We try 33 different delays and pick the one with the most energy; that delay corresponds to the direction of the source.

Q8. What is the maximum delay and how did you choose -16 to +16? Spacing 30.5 mm / speed of sound 343 m/s = 89 µs = 17.4 samples at 195.3 kHz. A range of ±16 samples covers about ±67°, which is nearly the full half-circle in front of the array.

Q9. Why do you square the summed signal? Energy (power) is proportional to amplitude squared. Squaring also makes positive and negative samples both count, so the sum over 512 samples measures how strong the combined wave is.

Q10. Why the ring buffer? To delay a signal we need its past samples. The ring buffer keeps the last 1024 samples (5.2 ms) of each mic in block RAM; reading at a different address gives a different delay without moving any data.

Q11. Why take log2 of the energy? Energy varies over a huge range. A log scale turns ratios into differences (1 step = 3 dB), so the colours can be defined as "how many dB below the loudest direction". It is done cheaply by finding the highest 1 bit plus 3 bits after it.

Q12. What does the noise gate do? If even the loudest direction is below a set level (15 + SW5–SW2), the image is switched off (all blue) and NEAR shows --. Otherwise background noise would show as random sources.

Q13. How is the nearest microphone decided? From the peak direction: if the best delay says the sound reached Mic 1 first (peak in the left part, column < 15) the nearest is Mic 1; if Mic 2 first (column > 17) it is Mic 2; within one column of the centre it is EQ.

Q14. Why does a low tone show EQ or a wide red area? With mics only 3 cm apart, a low frequency has a long wavelength (17 cm at 2 kHz), so the phase difference between mics is small and every direction gives almost the same energy. Higher tones (3–5 kHz) give a clearer difference. This is the array resolution limit, not a bug.

Q15. What limits the highest usable frequency? Spatial aliasing: when the mic spacing is more than half a wavelength (above about 5.6 kHz for 30.5 mm), the pattern repeats and directions become ambiguous. The low-pass filter also reduces signals above about 4 kHz.

Q16. How is the frequency measured? By counting upward zero crossings for exactly 1 second, with hysteresis (the signal must swing past ±peak/4) so noise is not counted. The count is kept in BCD so it can be displayed directly.

Q17. How is the VGA signal generated without a video chip? Two counters scan 800 x 525 positions at 25 MHz; HSYNC and VSYNC pulses are produced at fixed counter values; for every visible pixel combinational logic chooses a colour, which goes to the ADV7123 DAC on the board.

Q18. How is the text drawn? The screen is divided into 8 x 8 pixel cells. A case table gives the character in each cell, and a small 5 x 7 font table gives which pixels of that character are lit.

Q19. Why do you use clock enables instead of separate clocks? Everything runs on one 50 MHz clock; slower rates (3.125 MHz, 195 kHz, 25 MHz pixels) are made with one-cycle enable pulses. One clock domain avoids clock-domain-crossing problems and makes timing analysis simple.

Q20. What does the image show vertically? Time. Each row is the direction map at one moment; a new row is added at the top every 22 ms and older rows move down, so about 1 s of history is visible. A moving source draws a track.

Q21. How did you verify the design? In ModelSim with a testbench that models PDM mics with realistic timing and a source that moves from one side to the other, then on the board step by step: first one mic with LEDs and 7-segment displays (density, level, frequency), then two mics, then the full VGA system.

Q22. What FPGA resources are used? Block RAM for the two ring buffers (1024 x 16 bits each) and the image memory (two copies of 4096 x 8 bits), a few multipliers (DSP blocks) for squaring and blending, and logic for the counters and filters. Check the exact numbers in the Quartus Fitter report before the review.

Q23. How would you extend it to a 2-D image? Add more mics in a 2-D arrangement (for example 4 in a square, as in the reference paper's 32-mic array) and steer in both x and y; more mics and larger spacing also give a narrower beam and a sharper image.

Limitations, future work and glossary

Saying the limits yourself shows you understand the physics; examiners usually ask about them.

Limitations

- **Left/right only.** Two mics on a line can only tell left from right (one angle). There is no up/down information; the vertical direction of the image is time, not space.
- **Wide beam.** With 30.5 mm spacing the beam is very wide, so the red area is large and low tones (below about 2 kHz) look almost the same from every direction (often EQ).
- **Frequency range.** Best between about 2 and 5 kHz; above about 5.6 kHz spatial aliasing makes directions ambiguous, and the low-pass filter weakens signals above about 4 kHz.
- **Room echoes.** Reflections from walls and the desk arrive from other directions and can shift or widen the peak.

- **Integer delays.** Delays are whole samples (5.1 µs), so the direction is quantised into 33 steps.
- **Frequency update.** The frequency is measured over 1 s, so FREQ updates once per second and assumes one dominant tone.
- **Legend.** The INTENSITY bar on the screen is a fixed colour key, not a live measurement.

Future work

- 4 or more mics in a 2-D layout for a real 2-D acoustic image (as in the reference paper).
- Wider spacing or more mics for a narrower beam; fractional delays (interpolation) for finer direction steps.
- An FFT so the image can be made for one chosen frequency band.
- Overlaying the sound image on a camera picture.

Glossary

Term	Meaning
PDM	Pulse density modulation: audio as a fast stream of 1-bit values
PCM	Pulse code modulation: normal audio, one multi-bit number per sample
CIC	Cascaded integrator-comb filter, a multiplier-free decimation filter
Decimation	Reducing the sample rate (here by 16)
Beamforming	Combining several mics with delays to listen in one direction
Delay-and-sum	The simplest beamformer: delay each mic, then add
Ring buffer	Memory written in a circle, always holding the latest N samples
dB	Decibel; +3 dB = double the energy, +6 dB = double the amplitude
Noise gate	A threshold below which the output is switched off
BCD	Binary-coded decimal, one decimal digit per 4 bits

Term	Meaning
Clock enable	A one-cycle pulse that lets a register update at a slower rate on the main clock
Metastability	An unstable flip-flop state when the input changes at the clock edge
HSYNC / VSYNC	VGA timing pulses for end of line and end of frame
DAC	Digital-to-analog converter (ADV7123 on the DE2-115 for VGA)
Spatial aliasing	Direction ambiguity when mic spacing exceeds half a wavelength